

# **Google Cloud Functions (GCF)**

## **INDEX / TABLE OF CONTENTS**

- 1. What is Google Cloud Functions?**
- 2. Why Do We Use Cloud Functions?**
- 3. What Does Serverless Mean?**
- 4. What Does Event-Driven Mean?**
- 5. Cloud Functions 1st Gen vs 2nd Gen**
- 6. Types of Cloud Function Triggers**
- 7. HTTP Trigger**
- 8. Cloud Storage Trigger**
- 9. Pub/Sub Trigger**
- 10. Important Cloud Function Files**
- 11. Service Account and Permissions**
- 12. Deploy the Function**
- 13. How to Check Logs**
- 14. Cloud Functions vs Other GCP Services**
- 15. Google Cloud Run Services Console**
- 16. Cloud Run Console Actions**
- 17. Cloud Run Services Table**
- 18. Deployment Source Selection**
- 19. Basic Configuration**
- 20. Triggers and Security**
- 21. Billing, Scaling and Networking**
- 22. Container and Resource Settings**
- 23. Execution Environment and Advanced Concurrency**
- 24. Cloud SQL and Networking**
- 25. Security and Encryption**
- 26. Deployment Status, Revisions and Source Editor**
- 27. Step-by-Step: Create a Cloud Function**

**28. Testing the Cloud Function**

**29. Troubleshooting**

**30. Conclusion**

## 1. What is Google Cloud Functions?

**Google Cloud Functions** is a serverless service in GCP that lets us run small pieces of code when something happens.

For example:

- A file is uploaded to Cloud Storage.
- A message arrives in Pub/Sub.
- Someone sends an HTTP request.
- A particular event happens in GCP.

Instead of continuously running a server, we write the function and Google Cloud runs it when required.

### Simple Example

Suppose a user uploads: **orders.csv** to a Cloud Storage bucket.

The flow can be:

User uploads file



Cloud Storage



Cloud Function is triggered



Function processes the file



Data is loaded into Big Query

## 2. Why Do We Use Cloud Functions?

Without Cloud Functions, we may need to create and maintain a server.

We would have to manage:

- Virtual machines
- Operating systems

- Server maintenance
- Scaling
- Infrastructure

With Cloud Functions, Google manages most of this for us.

### **Main Benefits**

- No server management
- Automatically scales
- Runs only when needed
- Good for small tasks
- Works well with other GCP services
- Easy to connect services together

### **Simple Example**

Imagine a light:

No person → Light OFF

Person enters → Light ON

Person leaves → Light OFF

Cloud Functions works in a similar way:

No event → Function not running

Event occurs → Function runs

Task completed → Function stops

## **3. What Does Serverless Mean?**

**Serverless does NOT mean there are no servers.**

Servers are still used, but **Google manages the servers for us.**

We don't have to worry about:

- Server creation
- Operating system
- Server scaling
- Server maintenance

We mainly focus on writing our application code.

### **Example**

Traditional approach:

Application



Virtual Machine



Operating System



Server Management

Serverless approach:

Application Code



Cloud Function



Google manages infrastructure

## **4. What Does Event-Driven Mean?**

**Event-driven means the function runs when a specific event happens.**

Examples of events:

- File uploaded to Cloud Storage
- Message published to Pub/Sub
- HTTP request received
- Database event occurs

### **Example**

CSV uploaded to GCS



Event generated



Cloud Function starts



CSV is processed

The function does not need to continuously check whether a file has arrived.

## 5. Cloud Run functions: 1st Gen vs 2nd Gen

Cloud Functions has two generations.

| Feature                                                                                                                                                                                       | 1st Gen                        | 2nd Gen                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|--------------------------------|
| Infrastructure                                                                                                                                                                                | Cloud Functions infrastructure | Cloud Run + Eventarc           |
| Maximum HTTP timeout                                                                                                                                                                          | 9 minutes                      | Up to 60 minutes               |
| Maximum memory                                                                                                                                                                                | 8 GB                           | 16 GiB                         |
| Maximum CPU                                                                                                                                                                                   | 2 vCPU                         | 4 vCPU                         |
| Concurrency                                                                                                                                                                                   | 1 request per instance         | Multiple requests per instance |
| Current terminology: Google Cloud now refers to Cloud Functions 2nd Gen as Cloud Run functions.<br>For current limits, verify the latest Cloud Run functions documentation before deployment. |                                |                                |
| Event handling                                                                                                                                                                                | GCP triggers                   | Eventarc                       |

### Why is 2nd Gen important?

2nd Gen provides:

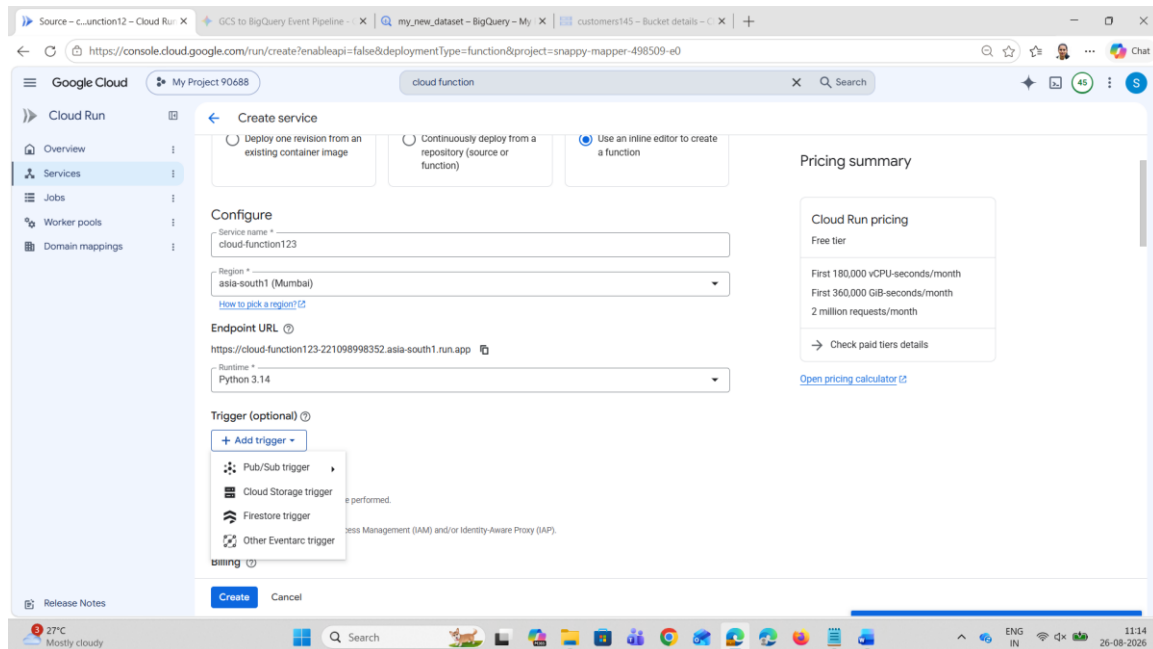
- Better scalability
- More memory
- More CPU
- Longer execution time
- Concurrency
- Better event integration

For new projects, **2nd Gen is generally preferred.**

## 6. Types of Cloud Function Triggers

A **trigger** tells Cloud Functions when to execute.

There are three important types to understand.



## 7. HTTP Trigger

An HTTP trigger runs when someone sends an HTTP request to the function.

For example:

Application



HTTP POST request



Cloud Function



Process request



Return response

### When do we use it?

Use HTTP triggers when another application needs to directly call our function.

Examples:

- REST APIs
- Webhooks
- Applications
- Third-party systems

### Example

A CRM system sends customer data:

```
{  
  "customer_id": "C101",  
  "name": "Harshitha"  
}
```

The Cloud Function receives the request and processes it.

### Important Point

HTTP functions should generally be used for **short operations**.

Do not use them for very long-running processing because the caller may receive a timeout.

## 8. Cloud Storage Trigger

A Cloud Storage trigger runs when something happens to a file in a bucket.

For example:

File uploaded



Cloud Storage



Cloud Function triggered



Process file

### Common Use Case

Suppose we have: gs://raw-data-bucket/orders.csv

When orders.csv is uploaded:

Cloud Storage



Cloud Function



Validate CSV



Load into BigQuery

### **Better Approach**

Use separate buckets:

Raw Bucket



Cloud Function



Processed Bucket

## **9. Pub/Sub Trigger**

**Pub/Sub** is used to send messages between applications and services.

Example:

Application



Pub/Sub Topic



Cloud Function

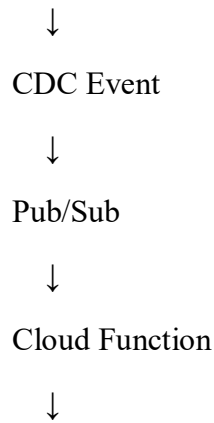


Process Message

### **Data Engineering Example**

Suppose a database sends change events.

Database



Process Event

This is useful for real-time or asynchronous processing.

## 10. Important Cloud Function Files

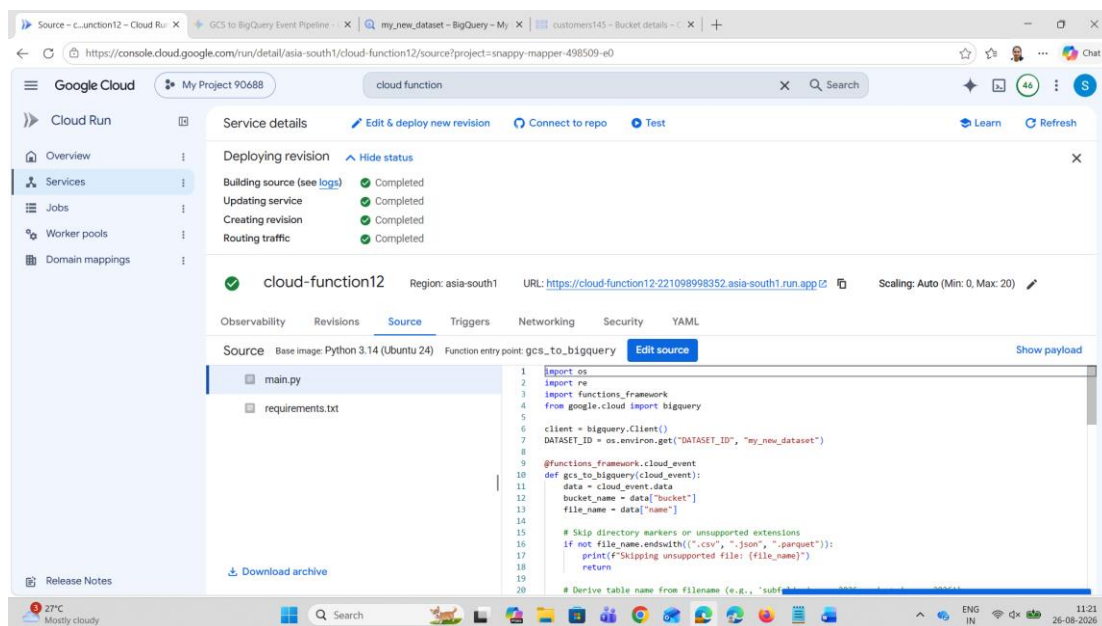
A Python Cloud Function normally contains:

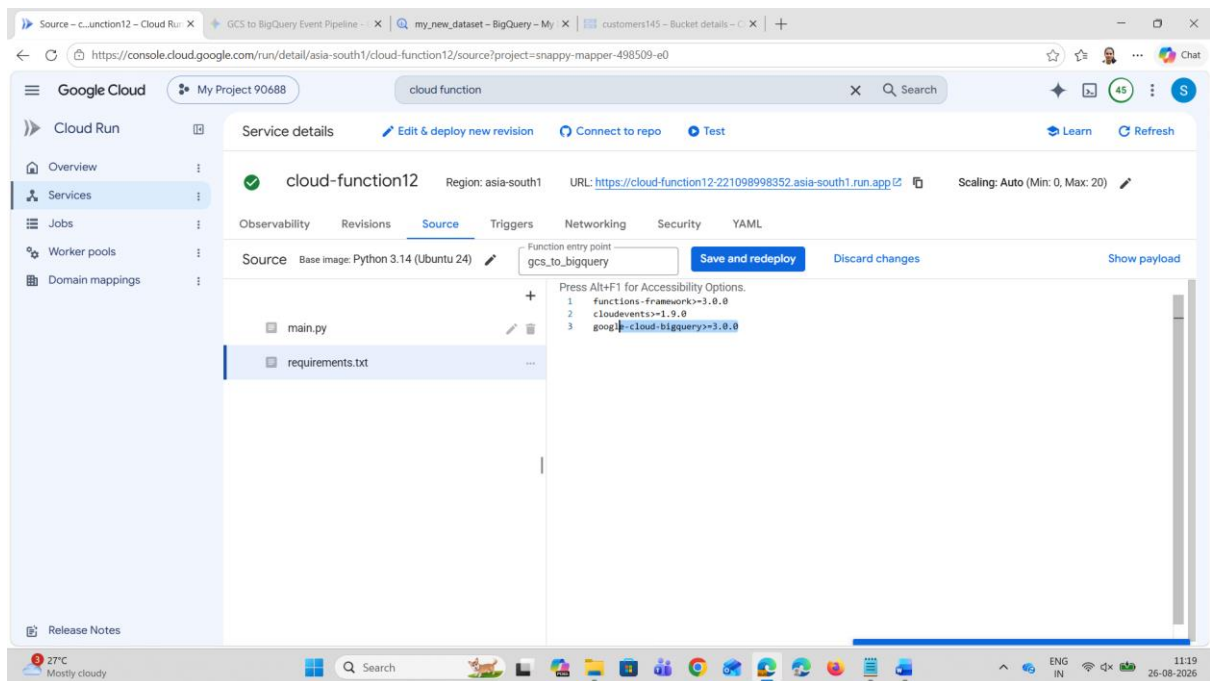
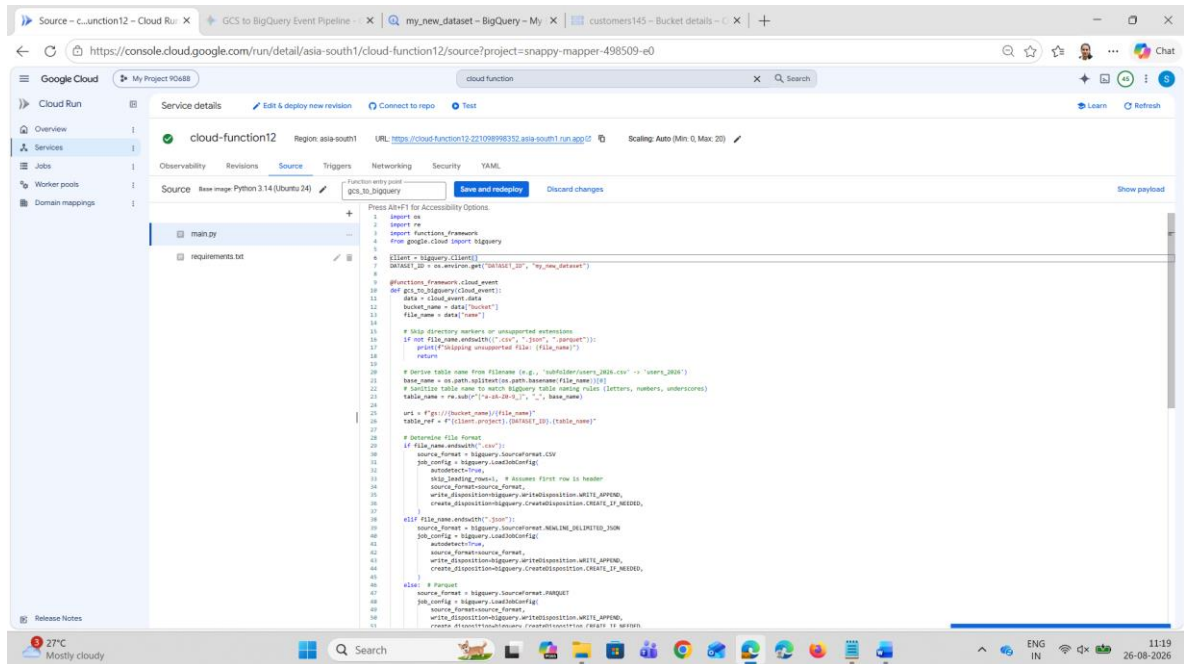
my-function/

├── main.py

├── requirements.txt

└── .env.yaml (optional)





## **main.py**

Contains the Python code.

Example:

```
import os
import re
import functions_framework
from google.cloud import bigquery

client = bigquery.Client()

DATASET_ID = os.environ.get("DATASET_ID", "my_new_dataset")

@functions_framework.cloud_event
def gcs_to_bigquery(cloud_event):
    data = cloud_event.data
    bucket_name = data["bucket"]
    file_name = data["name"]

    # Skip directory markers or unsupported extensions
    if not file_name.endswith(("csv", "json", "parquet")):
        print(f"Skipping unsupported file: {file_name}")
        return

    # Derive table name from filename (e.g., 'subfolder/users_2026.csv' -> 'users_2026')
    base_name = os.path.splitext(os.path.basename(file_name))[0]

    # Sanitize table name to match BigQuery table naming rules (letters, numbers,
underscores)
    table_name = re.sub(r"[^a-zA-Z0-9_]", "_", base_name)

    uri = f"gs://{bucket_name}/{file_name}"
```

```

table_ref = f"{client.project}.{DATASET_ID}.{table_name}"

# Determine file format
if file_name.endswith(".csv"):
    source_format = bigquery.SourceFormat.CSV
    job_config = bigquery.LoadJobConfig(
        autodetect=True,
        skip_leading_rows=1, # Assumes first row is header
        source_format=source_format,
        write_disposition=bigquery.WriteDisposition.WRITE_APPEND,
        create_disposition=bigquery.CreateDisposition.CREATE_IF_NEEDED,
    )
elif file_name.endswith(".json"):
    source_format = bigquery.SourceFormat.NEWLINE_DELIMITED_JSON
    job_config = bigquery.LoadJobConfig(
        autodetect=True,
        source_format=source_format,
        write_disposition=bigquery.WriteDisposition.WRITE_APPEND,
        create_disposition=bigquery.CreateDisposition.CREATE_IF_NEEDED,
    )
else: # Parquet
    source_format = bigquery.SourceFormat.PARQUET
    job_config = bigquery.LoadJobConfig(
        source_format=source_format,
        write_disposition=bigquery.WriteDisposition.WRITE_APPEND,
        create_disposition=bigquery.CreateDisposition.CREATE_IF_NEEDED,
    )

print(f>Loading {uri} into table {table_ref} (creates table automatically if not exists)...)

```

```
load_job = client.load_table_from_uri(uri, table_ref, job_config=job_config)

load_job.result() # Waits for BigQuery job completion


print(f'Success: Loaded {load_job.output_rows} rows into
`{DATASET_ID}.{table_name}`.')

```

### **requirements.txt**

Contains the Python packages required by the function.

functions-framework

google-cloud-storage

google-cloud-bigquery

Google installs these packages when the function is deployed.

Example:

functions-framework>=3.0.0

cloudevents>=1.9.0

google-cloud-bigquery>=3.0.0

## **11. Service Account and Permissions**

A **Service Account** is an identity used by applications or services.

Our Cloud Function needs permission to:

- Read files from Cloud Storage
- Write/load data into BigQuery

We should give only the permissions that are required.

### **Understanding WRITE\_APPEND**

In the example:

write\_disposition=bigquery.WriteDisposition.WRITE\_APPEND

### **WRITE\_APPEND**

Add new data.

## **WRITE\_TRUNCATE**

Remove existing table data and replace it with new data.

## **WRITE\_EMPTY**

Load data only if the destination table is empty.

## **12. Deploy the Function**

Example:

```
gcloud functions deploy gcs-to-bq-loader \
  --gen2 \
  --runtime=python311 \
  --region=${REGION} \
  --source=. \
  --entry-point=load_gcs_to_bigquery \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-event-filters="bucket=${RAW_BUCKET}"
```

Important options:

### **--gen2**

Deploy Cloud Functions 2nd Gen.

### **--runtime**

Specifies the Python version.

### **--source**

Specifies where the code is located.

### **--entry-point**

Specifies which Python function should run.

### **--trigger-event-filters**

Defines when the function should execute.

### 13. How to Check Logs

Use:

```
gcloud functions logs read gcs-to-bq-loader \  
  --region=${REGION} \  
  --gen2 \  
  --limit=20
```

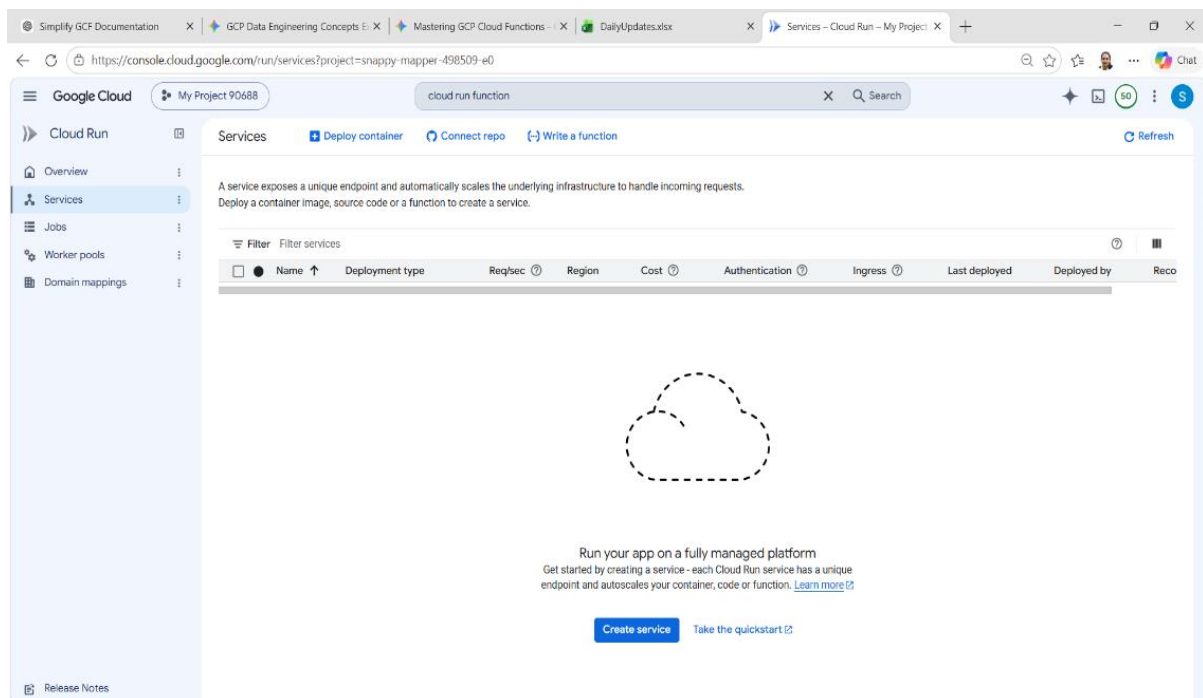
Logs help us understand:

- Whether the function started
- Which file was received
- Whether an error occurred
- Whether BigQuery loading succeeded

### 14. Cloud Functions vs Other GCP Services

| Service                               | Simple meaning                         | Best suited for                                       |
|---------------------------------------|----------------------------------------|-------------------------------------------------------|
| Cloud Functions / Cloud Run functions | Small event-driven serverless code.    | File triggers, webhooks, lightweight event processing |
| Cloud Run                             | Run containerized applications.        | APIs, microservices, custom applications              |
| Dataflow                              | Large-scale managed data processing.   | Batch and streaming pipelines                         |
| Cloud Composer                        | Managed workflow orchestration.        | Complex DAGs and scheduled workflows                  |
| Dataproc                              | Managed Spark/Hadoop.                  | PySpark, Spark, Hadoop big-data processing            |
| Cloud Run Jobs                        | Run containers as tasks that complete. | Batch jobs and scheduled/triggered container tasks    |

## 15. Google Cloud Run Services Console:



### Explain about the image in detail

#### 1. Overview

The **Cloud Run Services** page is used to view, create, and manage applications running on Google Cloud Run.

Google Cloud Functions **2nd Gen** is integrated with the Cloud Run product family as **Cloud Run functions**. This is because 2nd Gen functions run using managed container infrastructure provided by Cloud Run.

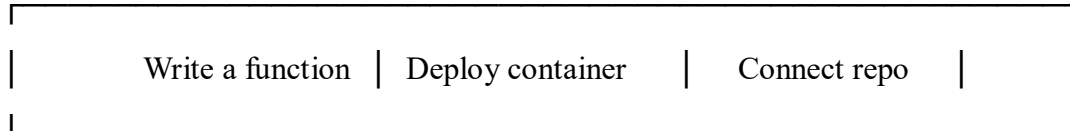
The Cloud Run Services page provides options to:

- Create a Cloud Run function
- Deploy a container
- Connect a source-code repository

## 16.Cloud Run Console Actions

### 16.1 Core Action Buttons

At the top of the Cloud Run Services page, there are three important options:



### 16.2Write a Function

#### What is it?

**Write a function** is used to create a Cloud Function / Cloud Run function from source code.

Typically, we provide:

main.py

requirements.txt

Google Cloud builds and deploys the function for us.

#### Why is it used?

It is useful when we want to run a small piece of code automatically when an event occurs.

Common examples:

- Cloud Storage file upload
- Pub/Sub message
- HTTP request
- Lightweight data-processing task
- Webhook processing

#### How does it work?

Write Python Code



Provide main.py



Provide requirements.txt



Google Cloud builds the application



Cloud Run function is deployed



Function waits for the trigger



Event occurs



Function executes

### **When should we use it?**

Use **Write a function** when the application is mainly a small, event-driven task.

## **16.3 Deploy Container**

### **What is it?**

**Deploy container** is used to deploy an already-built container image to Cloud Run.

The container image can be stored in **Artifact Registry**.

### **Why is it used?**

Use this option when the application requires a custom container environment.

Examples include:

- Custom system libraries
- Custom runtime requirements
- Compiled libraries
- Docker configuration
- FastAPI applications
- Django applications
- Custom microservices

### **Simple Difference**

Write a function



Provide source code



Google manages the container setup

Deploy container



Provide your own container image



Cloud Run runs the container

## 16.4 Connect Repo

### What is it?

**Connect repo** connects Cloud Run to a source-code repository such as:

- GitHub
- GitLab
- Bitbucket

It can be used to set up automated deployments.

### Why is it used?

It helps implement **CI/CD**.

For example:

Developer writes code



Code pushed to GitHub



Build process starts



Application is built



New version is deployed

### **When should we use it?**

It is useful for production environments where:

- Code is maintained in Git
- Developers use pull requests
- Code needs review
- Deployments should be automated

## **3. Left Navigation Menu**

The Cloud Run console contains several sections.

Cloud Run

|

├— Overview

├— Services

├— Jobs

├— Worker pools

└— Domain mappings

### **Overview**

Provides a general summary of Cloud Run resources and information.

### **Services**

Used to view and manage deployed Cloud Run services and functions.

### **Jobs**

Used for tasks that run to completion.

Example:

Start Job

↓

Process Data

↓

Task Completed

↓

Job Ends

### Worker Pools

Used for background worker workloads.

### Domain Mappings

Used to connect a custom domain to a Cloud Run service.

For example:

api.example.com



Cloud Run Service

## 4. Services Table

When we open **Cloud Run** → **Services**, the console displays information about deployed services and functions.

The table contains several important columns.

| Column Name            | What It Is                                    | Why / How It Is Used                              |
|------------------------|-----------------------------------------------|---------------------------------------------------|
| <b>Name</b>            | Unique name of the service or function        | Used to identify the resource                     |
| <b>Deployment type</b> | Shows whether it is a Function or Container   | Helps identify how the application was deployed   |
| <b>Req/sec</b>         | Requests handled per second                   | Helps monitor traffic and load                    |
| <b>Region</b>          | GCP location where the resource runs          | Helps with latency and data-location requirements |
| <b>Cost</b>            | Estimated cost information for the service    | Helps monitor spending                            |
| <b>Authentication</b>  | Shows whether authentication is required      | Controls who can access the service               |
| <b>Ingress</b>         | Controls where incoming traffic can come from | Helps control network access                      |
| <b>Last deployed</b>   | Shows the latest deployment information       | Useful for tracking deployments and revisions     |

|                    |                                                    |                                  |
|--------------------|----------------------------------------------------|----------------------------------|
| <b>Deployed by</b> | Shows the user or service account that deployed it | Useful for auditing and security |
|--------------------|----------------------------------------------------|----------------------------------|

## 17. Services Table Columns in Detail

### 17.1 Name

The **Name** is the unique identifier of the Cloud Run service or function.

Example:

[gcs-to-bq-loader](#)

It helps us identify and manage the deployed resource.

### 17.2 Deployment Type

This shows how the application was deployed.

Common values include:

Function

Container

#### **Function**

The application was deployed as a Cloud Run function.

#### **Container**

The application was deployed using a container image.

### 17.3 Req/sec

**Req/sec** means **requests per second**.

It shows how many requests the service is handling per second.

Example:

100 req/sec

means approximately 100 requests are being handled every second.

This is useful for monitoring traffic and identifying load spikes.

## 17.4 Region

The **Region** shows the GCP location where the service is deployed.

Examples:

us-central1

asia-south1

The region can affect:

- Network latency
- Data location
- Compliance requirements
- Application performance

## 17.5 Cost

The **Cost** column provides estimated cost information for the service.

It helps us:

- Monitor spending
- Understand resource usage
- Identify expensive services
- Manage cloud budgets

## 17.6 Authentication

Authentication controls **who can access the service**.

Two common options are:

### **Allow unauthenticated**

The service can be accessed publicly.

Internet



Cloud Run Service

### **Require authentication**

Only authorized users or services can access it.

Authorized User / Service



Cloud Run

### **Simple Meaning**

**Authentication = Who is allowed to access the service?**

## **17.7 Ingress**

**Ingress** controls where incoming traffic is allowed to come from.

Common options are:

### **All**

Allows traffic from external and internal sources.

### **Internal**

Restricts access to internal sources.

### **Internal + Cloud Load Balancing**

Allows traffic through Cloud Load Balancing while maintaining network restrictions.

### **Simple Meaning**

Authentication = Who can access?

Ingress = Where can traffic come from?

## **17.8 Last Deployed**

This shows information about the most recent deployment.

It is useful for:

- Tracking deployments
- Checking when a change was made
- Identifying the latest revision
- Troubleshooting deployment-related issues
- Rollback planning

## 17.9 Deployed By

This shows the user or service account that performed the deployment.

It is useful for:

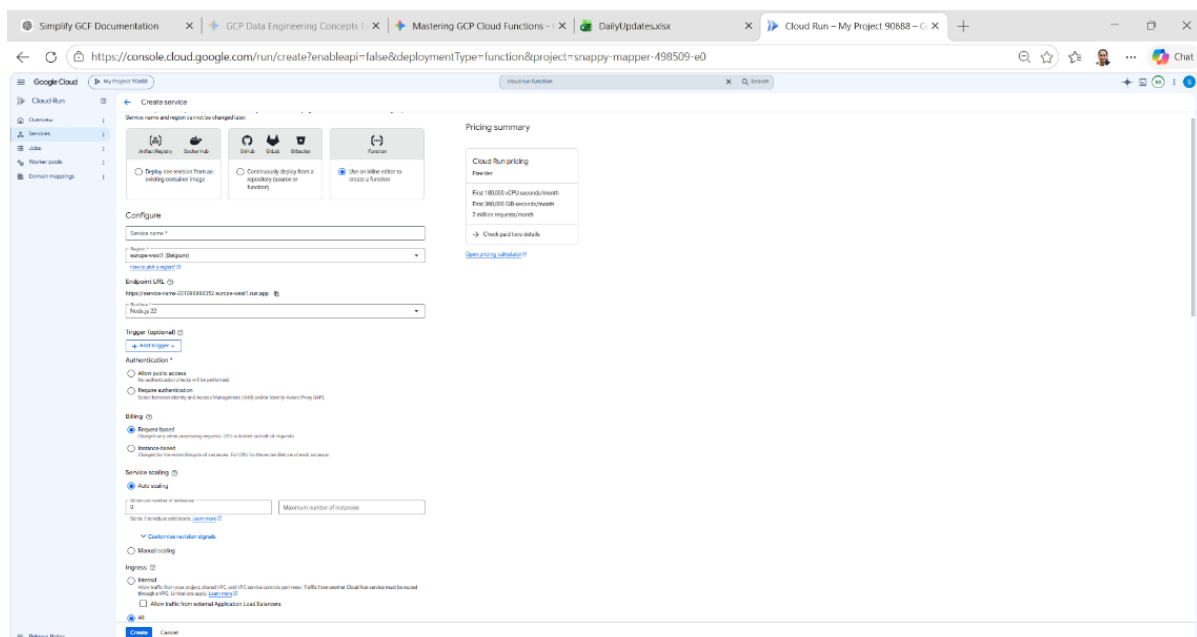
- Auditing
- Security
- Troubleshooting
- Tracking deployment changes

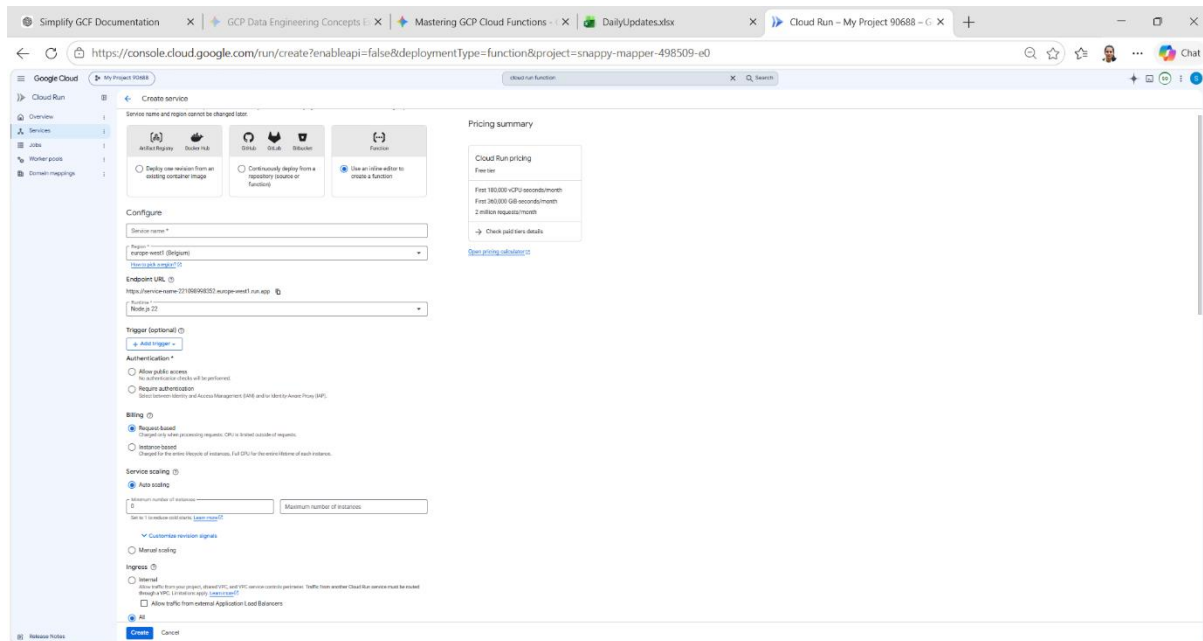
## 18. Deployment Source Selection

The Cloud Run creation console provides different ways to deploy an application or function.

[ Artifact Registry / Docker Hub ]    [ GitHub / GitLab / Bitbucket ]    [ Function ]

Deploy existing container image      Continuously deploy from repo      Use an inline editor





The maximum number of concurrent requests that can reach each instance. [What is concurrency?](#)

**Execution environment**  
The execution environment your container runs in. [Learn More](#)

☒ **Default**  
Cloud Run will select a suitable execution environment for you.

☐ **First generation**  
Faster cold starts.

☐ **Second generation**  
Network file system support, full Linux compatibility, faster CPU and network performance.

**Revision scaling** ⓘ  
Minimum and maximum numbers of instances for the new revision.

Minimum number of instances:  Maximum number of instances:

The service minimum instances is preferable for most use-cases. Only use this setting if you specifically require per-revision settings.

☒ **Startup CPU boost**  
Start containers faster by allocating more CPU during startup time. [Learn more](#)

**Cloud SQL connections** ⓘ

[+ Add connection](#)

[Create](#) [Cancel](#)

## 18.1 Function – Use an Inline Editor to Create a Function

### What?

The **Function** option provides a serverless source-code workflow for creating a function without manually managing a container image.

It represents the modern Cloud Run-based approach to deploying serverless functions, which were previously provided as standalone Cloud Functions.

## Why?

It is useful when you want to write a **single-purpose function** using languages such as:

- Python
- Node.js
- Go

You do not need to manually create:

- A Dockerfile
- A container image
- A local container build process
- A container registry workflow

## How?

You provide source files such as:

main.py

requirements.txt

Google Cloud uses **Cloud Build and Buildpacks** behind the scenes to:

1. Detect the programming language.
2. Install the required dependencies.
3. Build the application.
4. Create a container image.
5. Deploy the container to Cloud Run.

## 18.2 Deploy One Revision from an Existing Container Image

### What?

This is the direct container deployment option.

You provide an already-built container image from a container registry such as:

- Artifact Registry
- Docker Hub

### Why?

This approach is useful when:

- You already have a container image.
- Your application requires custom system libraries.
- You need custom binaries such as ffmpeg.
- You require a specific operating-system configuration.
- Your application is already packaged as a Docker container.

### **How?**

The process is:

Container Image



Artifact Registry / Docker Hub



Cloud Run



Container Instance

Cloud Run pulls the existing image and runs it as a service.

## **18.3 Continuously Deploy from a Repository**

### **What?**

This option connects Cloud Run to a source-code repository such as:

- GitHub
- GitLab
- Bitbucket

It supports automated deployment from source control.

### **Why?**

It is useful when you want to implement a **CI/CD workflow** and avoid manually deploying every code change.

For example:

Developer



Git Push



GitHub / GitLab / Bitbucket



Cloud Build



Build Container



Cloud Run



New Revision

### **How?**

A repository is connected to Google Cloud, and changes pushed to the configured branch can trigger an automated build and deployment process.

This allows teams to maintain a continuous deployment workflow.

## **19. Basic Configuration**

### **19.1 Service Name**

#### **What?**

The **Service name** is the unique name assigned to the Cloud Run service within the project and region.

#### **Why?**

The service name helps identify and manage the deployed service.

Cloud Run also uses the service name as part of the default service URL.

Example:

`https://SERVICE-NAME-<PROJECT-HASH>.<REGION>.run.app`

#### **How?**

The name generally follows Cloud Run naming requirements, including:

- Lowercase letters
- Numbers

- Hyphens
- Maximum length of 63 characters

Choose a meaningful name such as:

customer-data-processor

order-processing-api

file-upload-function

The service name cannot simply be changed after deployment; changing it generally requires creating another service.

## 19.2 Region

### What?

The **Region** specifies the Google Cloud geographical region where the Cloud Run service is deployed.

Example:

eu-west-1 (Belgium)

### Why?

The selected region affects:

- Network latency
- Data residency
- Compliance requirements
- Availability
- Network/data-transfer costs
- Proximity to dependent services

### How?

Select a region close to your users and major dependent services.

For example:

Cloud Run

↓

Cloud Storage

↓

## BigQuery

Keeping services in compatible or appropriate locations can help reduce latency and avoid unnecessary inter-region data-transfer costs.

### 19.3 Endpoint URL

#### What?

The **Endpoint URL** is the HTTPS address through which the Cloud Run service can receive requests.

#### Why?

It provides a managed HTTPS endpoint without requiring you to manually configure a web server or SSL certificate for the default URL.

#### How?

After deployment, Google Cloud provides a service URL similar to:

`https://service-name-xxxxxx.region.run.app`

Incoming HTTPS traffic is handled by Google's managed infrastructure and routed to the appropriate Cloud Run service.

### 19.4 Runtime

Examples include:

Python 3.11

Python 3.12

Node.js 22

#### What?

The **Runtime** specifies the programming language environment used by the function.

#### Why?

It determines the language environment and build configuration required to execute your source code.

For example:

- Python → requirements.txt
- Node.js → package.json
- Go → Go module configuration

## How?

Google Cloud uses the selected runtime during the build and deployment process.

For example:

main.py

requirements.txt



Cloud Build / Buildpacks



Python Runtime



Container Image



Cloud Run

## 20. Triggers & Security

### 20.1 Trigger – Add Trigger

#### What?

A **Trigger** allows the function to execute automatically when a specific event occurs.

Event-driven functions can use services such as **Eventarc** to receive supported Google Cloud events.

#### Why?

Triggers are useful when the function should run automatically rather than waiting for a user to call an HTTP endpoint.

Examples include:

- A file is uploaded to Cloud Storage.
- A message is published to Pub/Sub.
- A Google Cloud event occurs.

#### How?

The basic flow is:

Event Source



Eventarc



Cloud Run Function



Function Code

The event is delivered to the function, which processes the event.

## 20.2 Authentication

Authentication determines who is allowed to invoke the Cloud Run service.

### **Allow Public Access – Unauthenticated**

#### **What?**

Allows users or systems to invoke the service without providing Google Cloud authentication credentials.

#### **Why?**

Useful for services that intentionally need public access, such as:

- Public APIs
- Public webhooks
- Public endpoints
- Certain third-party webhook integrations

#### **How?**

The service is configured to allow unauthenticated invocation.

The public principal is represented by:

`allUsers`

The required Cloud Run invocation permission is associated with:

`roles/run.invoker`

## **Require Authentication**

### **What?**

Restricts access so that only authenticated and authorized callers can invoke the service.

### **Why?**

This is commonly preferred for:

- Internal APIs
- Backend services
- Enterprise applications
- Data-processing services
- ETL workloads

### **How?**

The caller must authenticate and provide an appropriate identity token.

A request can include an authorization header such as:

Authorization: Bearer <TOKEN>

Google Cloud validates the identity and permissions before allowing the request to reach the service.

## **21. Billing, Scaling & Networking**

### **21.1 Billing Modes**

Cloud Run provides different CPU allocation/billing behaviors depending on the selected configuration.

#### **Request-Based Billing**

##### **What?**

CPU is primarily allocated while the instance is actively processing requests.

##### **Why?**

It is useful for applications where work occurs mainly during incoming requests or events.

This can reduce costs for services that are idle for long periods.

##### **How?**

The general lifecycle is:

Request arrives



Instance processes request



CPU allocated



Request completes



CPU may be throttled

When the service has no active requests, instances can scale down, potentially reaching zero when configured to do so.

## **Instance-Based Billing**

### **What?**

CPU remains allocated for the lifetime of the running instance.

### **Why?**

Useful for workloads that need CPU outside the normal request-processing period, such as:

- Background processing
- Long-running background tasks
- Certain WebSocket workloads
- Applications that perform work continuously

### **How?**

The instance continues receiving allocated CPU while it remains active.

This can result in higher costs compared with request-based allocation when instances remain active for long periods.

## **21.2 Service Scaling**

Cloud Run can automatically adjust the number of running instances based on workload.

## Auto Scaling

### What?

Cloud Run automatically increases or decreases the number of instances according to demand.

### Why?

It allows the service to handle changing traffic without manually starting or stopping servers.

### How?

For example:

Low Traffic



1 instance

High Traffic



5 instances

Very High Traffic



10 instances

The actual number depends on configuration and available capacity.

## Minimum Number of Instances

### What?

The **minimum number of instances** specifies the minimum number of container instances Cloud Run should keep available.

### Why?

Setting the minimum to 0 allows the service to scale to zero when there is no traffic.

Setting it to 1 or more can keep instances warm and help reduce cold-start latency.

### How?

Example:

`min-instances = 0`

The service can scale to zero.

min-instances = 1

At least one instance is maintained when possible.

## Maximum Number of Instances

### What?

The **maximum number of instances** sets an upper limit on how many instances Cloud Run can create.

### Why?

It can help:

- Protect downstream databases.
- Prevent excessive concurrent processing.
- Control resource consumption.
- Reduce the risk of unexpected cost increases.

For example:

max-instances = 10

The service will not intentionally scale beyond the configured maximum under normal Cloud Run autoscaling behavior.

### How?

Example:

Traffic

↓

Cloud Run Autoscaler

↓

Maximum = 10

↓

Up to 10 instances

## **Manual Scaling**

### **What?**

Manual scaling maintains a configured number of instances instead of automatically scaling based on request volume.

### **Why?**

It may be useful for workloads that require a predictable number of running instances.

### **How?**

The number of running instances is explicitly configured rather than dynamically adjusted based on traffic.

## **21.3 Ingress Control**

Ingress controls where incoming traffic to the Cloud Run service can originate.

### **All**

#### **What?**

Allows traffic from public sources as well as permitted internal sources.

#### **Why?**

Useful when the service must be publicly accessible.

#### **How?**

Requests can reach the service through its public endpoint, subject to the service's authentication and authorization settings.

### **Internal**

#### **What?**

Restricts the service so that it is accessible only through supported internal Google Cloud networking paths.

#### **Why?**

Useful for internal applications and backend services that should not be directly exposed to the public internet.

#### **How?**

Traffic is restricted according to Cloud Run's internal ingress rules and supported Google Cloud networking configurations.

## **Internal + Cloud Load Balancing**

### **What?**

Restricts direct ingress while allowing traffic through a Cloud Load Balancer.

### **Why?**

Useful when organizations need additional networking and security capabilities such as:

- Cloud Armor
- Web Application Firewall policies
- Custom domains
- Load balancing
- CDN-related configurations

### **How?**

The traffic flow can be represented as:

User

↓

Cloud Load Balancer

↓

Security / Cloud Armor

↓

Cloud Run

## **21.4 Pricing Summary Box**

The console may display available free-tier allowances and estimated pricing information.

The displayed allowances can include:

- **180,000 vCPU-seconds** per month
- **360,000 GiB-seconds** of memory per month
- **2 million requests** per month

Always verify the current Google Cloud pricing page because pricing, free-tier limits, and eligibility can change.

## 22. Container & Resource Settings

The configuration area commonly contains sections or tabs such as:

Containers

Networking

Security

### 22.1 Container Port

**Default: 8080**

**What?**

The **Container Port** specifies the TCP port on which the application inside the container listens for incoming HTTP traffic.

**Why?**

Cloud Run needs to know which port should receive traffic inside the container.

**How?**

Cloud Run commonly uses the PORT environment variable.

For example:

PORT=8080

The application should listen on that port.

For a function deployment, the functions framework handles the server setup so that the function can receive requests through the expected port.

### 22.2 Resources – Memory

**What?**

**Memory** specifies the maximum amount of RAM available to a container instance.

Examples may include:

512 MiB

1 GiB

2 GiB

4 GiB

### **Why?**

Memory determines how much data the application can keep in memory while running.

Insufficient memory can result in an **Out-Of-Memory (OOM)** termination.

### **How?**

For example:

Application

↓

Uses 700 MiB

↓

Configured Memory = 512 MiB

↓

Insufficient Memory

↓

OOM termination

The appropriate memory size depends on the application's workload.

## **23.3 Resources – CPU**

### **What?**

**CPU** specifies the number of virtual CPUs assigned to a Cloud Run container instance.

Examples can include:

1 vCPU

2 vCPU

4 vCPU

8 vCPU

### **Why?**

CPU allocation affects the processing capability of the application.

CPU-intensive workloads may require more CPU.

### **How?**

For example:

1 vCPU



Suitable for lightweight workloads

4 vCPU



Suitable for more CPU-intensive workloads

The available CPU options depend on the selected Cloud Run configuration.

## 22.4 Ephemeral Disk

### What?

**Ephemeral disk** provides temporary local storage available to the running container instance.

### Why?

It can be useful when an application needs temporary disk space for:

- File processing
- Temporary downloads
- Intermediate files
- Data buffering

### How?

The application writes temporary files to the available local filesystem.

The storage is temporary and should **not** be treated as permanent storage.

For persistent data, use services such as:

Cloud Storage

Cloud SQL

BigQuery

## 22.5 GPU

### What?

A **GPU** provides hardware acceleration for supported Cloud Run workloads.

### **Why?**

GPUs are useful for workloads such as:

- AI inference
- Machine learning
- Large language model serving
- Image processing
- Video processing

### **How?**

Cloud Run schedules the workload on infrastructure with the requested GPU capability.

The application can then use the GPU through supported libraries and frameworks.

## **22.6 Health Checks**

### **What?**

Health checks determine whether the container is ready and functioning correctly.

Supported probe types can include:

- HTTP
- TCP
- gRPC

### **Why?**

Health checks help Cloud Run determine whether a container is ready to receive traffic and whether an unhealthy container should be restarted.

### **How?**

For example:

Cloud Run



Health Check



Application Health Endpoint



Healthy → Continue serving

Unhealthy → Take corrective action

Health checks are particularly useful for applications that require initialization such as:

- Loading machine-learning models
- Establishing connections
- Initializing application resources
- Loading large data structures

## **23. Execution Environment & Advanced Concurrency**

### **23.1 Request Timeout**

**Default: 300 seconds**

**Maximum: Up to 3600 seconds / 60 minutes for supported request configurations**

**What?**

The **Request Timeout** specifies the maximum amount of time allowed for an individual request or execution.

**Why?**

It prevents requests from running indefinitely.

**How?**

For example:

Request starts

↓

Application processes request

↓

Request completes

If the configured timeout is reached before completion, Cloud Run terminates the request.

The appropriate timeout depends on the workload.

## 23.2 Maximum Concurrent Requests per Instance

**Example: 80**

**What?**

**Concurrency** specifies the maximum number of requests that a single Cloud Run instance can process simultaneously.

**Why?**

Higher concurrency can improve resource utilization and reduce the number of instances required for lightweight workloads.

**How?**

For example:

Instance

|— Request 1

|— Request 2

|— Request 3

|— ...

|— Request 80

If concurrency is configured as 80, one instance can process up to 80 concurrent requests, subject to application behavior and Cloud Run limits.

For CPU-intensive workloads, lower concurrency may be more appropriate.

## 23.3 Execution Environment

Cloud Run supports different execution environments.

**First Generation**

**What?**

The first-generation execution environment provides a lightweight sandbox for running containers.

**Why?**

It can provide suitable performance and compatibility for many lightweight workloads.

**How?**

The workload runs within Google's managed sandbox environment.

## Second Generation

### What?

The second-generation execution environment provides a newer infrastructure model with improved Linux and networking compatibility.

### Why?

It is useful when an application requires:

- Broader Linux syscall compatibility
- Improved CPU performance
- Improved networking performance
- Features such as supported NFS/Cloud Filestore integration

### How?

Cloud Run runs the container using the second-generation execution environment and its underlying managed infrastructure.

## 23.4 Startup CPU Boost

### What?

**Startup CPU Boost** temporarily provides additional CPU resources while a new container instance is starting.

### Why?

It helps reduce startup time for applications that perform expensive initialization.

Examples include applications that load:

- Pandas
- NumPy
- TensorFlow
- Machine-learning models
- Large configuration files

### How?

The general process is:

New Instance



Startup CPU Boost



Faster Initialization



Application Ready



Normal CPU Allocation

This can help reduce cold-start latency.

## 24. Cloud SQL Connections

### What?

Cloud Run provides an integrated configuration for connecting a Cloud Run service to **Cloud SQL**.

Cloud SQL supports relational databases such as:

- PostgreSQL
- MySQL
- SQL Server

### Why?

It simplifies secure connectivity between Cloud Run and Cloud SQL without requiring the application team to manually implement all connection networking and authentication mechanisms.

### How?

The Cloud Run service is configured with the required Cloud SQL instance.

The application can then connect using supported Cloud SQL connection methods, including Unix sockets where applicable.

Example Unix socket path:

`/cloudsql/PROJECT:REGION:INSTANCE`

## 24. Networking

---Connect to other Google Cloud services like Google Cloud Storage or Google Cloud Firestore directly from your code.

- Use HTTP/2 end-to-end

---Use if your container is a gRPC streaming server or is able to directly handle requests in HTTP/2 cleartext.

- Session affinity

---Best effort to route requests from the same client to the same container instance.

- Connect to a VPC for outbound traffic

----Allow your service to send traffic to a Virtual Private Cloud. [How to choose between VPC options?](#)

- Join a Service Mesh.

---Allow your service to send traffic via Cloud Service Mesh.

## 25. Security

Service account ---Compute Engine default service account---Identity to be used by the created revision.

Verify container deployment with Binary Authorization

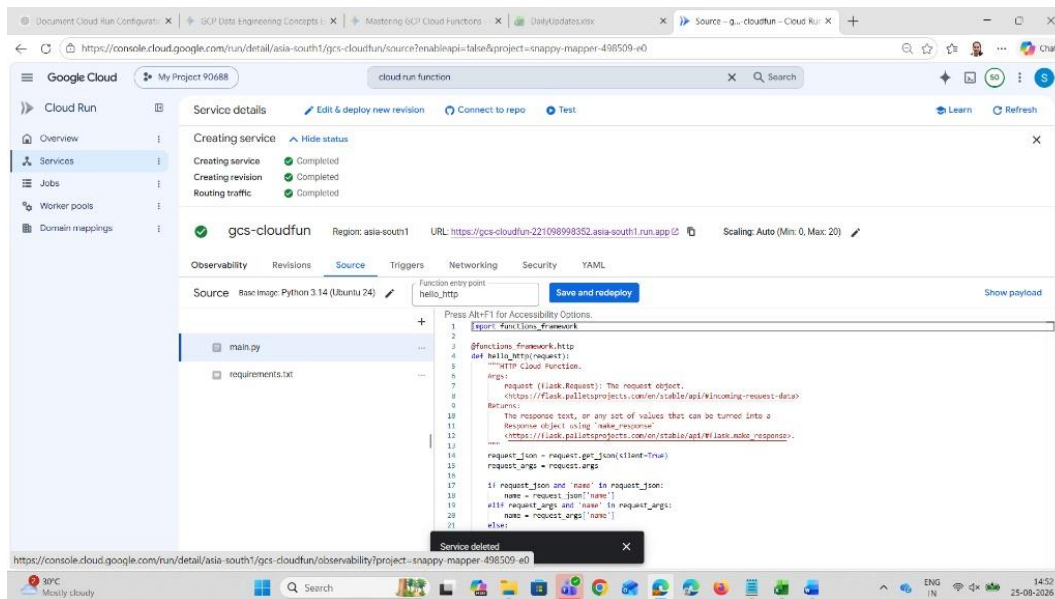
----Clicking the checkbox will enable the Binary Authorization API.

## 25. Encryption

Google-managed encryption key---Keys owned by Google

Cloud KMS key---Keys owned by customers

## 26. Deployment Status, Revisions and Source Editor



### 1. Top Action Toolbar

- **Edit & deploy new revision**
  - **What?** Opens the full configuration wizard (memory, CPU, environment variables, timeout, service account).
  - **Why?** Use this when you need to change hardware allocations or infrastructure settings rather than just changing lines of code.
  - **How?** Generates a brand-new immutable Revision container image and rolls traffic over to it.
- **Connect to repo**
  - **What?** Configures a continuous deployment (CI/CD) trigger via Cloud Build linked to GitHub, GitLab, or Bitbucket.
  - **Why?** Replaces manual in-console code editing with automated deployments from Git commits.
  - **How?** Creates a Cloud Build trigger that builds and deploys upon every git push.
- **Test**
  - **What?** An integrated testing utility to send mock HTTP requests or CloudEvents directly to the function inside the console.
  - **Why?** Allows instant debugging and verification without using external tools like Postman or curl.

- **How?** Sends a sample JSON payload directly to the authenticated endpoint URL.

## 2. Deployment Status Banner & Identity Bar

- **Creating service / Creating revision / Routing traffic (Completed)**
  - **What?** The deployment progress checklist.
  - **Why?** Verifies that Cloud Build successfully packaged the code, provisioned the microVM container revision, and shifted 100% of incoming live traffic to it.
  - **How?** Green checkmarks indicate that the underlying Knative/Cloud Run control plane successfully passed health checks.
- **Status Icon (Green Circle Checkmark)**
  - **What?** Service health status. Indicates the endpoint is ready to receive requests.
- **Service Name (gcs-cloudfun) & Region (asia-south1)**
  - **What?** Service identifier and physical data center location (**Mumbai, India**).
  - **Why?** Deploying in asia-south1 ensures low network latency for applications and data buckets located in India.
- **URL ([https://gcs-cloudfun-...-asia-south1.run.app](https://gcs-cloudfun-...-asia-south1.run.app))**
  - **What?** The dedicated HTTPS endpoint with automated SSL/TLS termination.
  - **Why?** Provides a secure web gateway to trigger the function.
- **Scaling: Auto (Min: 0, Max: 20)**
  - **What?** Autoscaling limits.
  - **Why?** Min: 0 ensures the service costs \$0 when idle (scales to zero). Max: 20 acts as a cost and database protection ceiling, preventing more than 20 container instances from running simultaneously during traffic spikes.

## 3. Console Navigation Tabs

- **Observability:** Real-time dashboards for Cloud Logging (stdout/stderr), invocation metrics, latency percentiles, memory/CPU utilization, and error traces.
- **Revisions:** Revisions can support controlled rollout and rollback strategies. Traffic can be shifted between revisions according to the deployment strategy.
- **Source (Active Tab):** The source editor can be used to view or modify files such as main.py and requirements.txt when that workflow is appropriate..

- **Triggers:** Manages Eventarc bindings, Cloud Storage events, and Pub/Sub subscriptions attached to this service.
- **Networking:** Configures Ingress access (Internal vs. Public), Serverless VPC Access connectors, and static outbound IP (Cloud NAT) settings.
- **Security:** Displays the assigned Service Account, Identity and Access Management (IAM) role permissions, and Secret Manager environment mounts.
- **YAML:** YAML can expose the declarative configuration of the deployed service and can be useful for understanding or exporting configuration for infrastructure workflows.

#### 4. Source Editor & Function Configuration Area

- **Base image: Python 3.14 (Ubuntu 24)**
  - **What?** The underlying operating system and language runtime container.
  - **Why?** Cloud Buildpacks use this base image to compile and run your Python scripts.
  - **How?** Click the pencil icon if you need to switch runtime versions (e.g., Python 3.11 or Python 3.12).
- **Function entry point (hello\_http)**
  - **What?** The exact Python function name inside main.py that Google Cloud executes upon receiving an incoming event or request.
  - **Why?** Tells the functions-framework web server which `@functions_framework` handler function to route traffic to.
  - **Common Mistake:** Changing the function definition name in main.py (e.g., `def my_etl_job(request):`) without updating this Function entry point field will crash the container with an initialization error.
- **Save and redeploy**
  - **What?** Primary execution trigger button.
  - **Why?** Applies any code changes made in the editor.
  - **How?** Triggers a new Cloud Build job, compiles a new container revision, and routes traffic to the updated code.
- **Show payload**
  - **What?** Opens a side drawer to view and customize sample JSON test payloads matching HTTP or CloudEvents schema formats.
- **File Explorer Pane (+, main.py, requirements.txt, ...)**

- **+ Button:** Adds new custom .py modules, utility scripts, or JSON schema files to your deployment directory.
- **main.py:** Main Python script containing the business logic and decorated entry point.
- **requirements.txt:** Manifest file declaring external PyPI libraries (such as google-cloud-bigquery, google-cloud-storage, pandas). Cloud Build runs pip install -r requirements.txt during deployment.
- **... (Three dots):** File action menu to rename or delete specific files.

## 27.Step-by-Step: Create a Cloud Function

### Step 1: Set Up Your Project and Enable APIs

1. **Select Your Project:** Open the Google Cloud Console and choose a dedicated practice project from the top project dropdown.
2. **Enable Required APIs:** Navigate to **APIs & Services > Library**, then search for and enable the following services:
  - Cloud Run API (run.googleapis.com)
  - Cloud Functions API (cloudfunctions.googleapis.com)
  - Eventarc API (eventarc.googleapis.com)
  - Cloud Build API (cloudbuild.googleapis.com)
  - Cloud Storage API (storage.googleapis.com)
  - BigQuery API (bigquery.googleapis.com)

### Step 2: Set Up Service Account and Storage Resources

1. **Create Storage & BigQuery Destinations:**
  - Go to **Cloud Storage > Buckets > Create** to create a bucket (e.g., gs://raw-orders-data).
  - Go to **BigQuery > select your project > Create Dataset** named sales\_data.
2. **Create a Service Account:**
  - Go to **IAM & Admin > Service Accounts > Create Service Account** (e.g., sa-gcs-bq-loader).

### 3. Grant Least-Privilege IAM Roles:

- Assign roles/storage.objectViewer (to read incoming files).
- Assign roles/bigquery.dataEditor (to write records to BigQuery).
- Assign roles/bigquery.jobUser (to run BigQuery load jobs).

### Step 3: Configure Function Settings in Console

1. Navigate to **Cloud Run** or **Cloud Functions** from the left-hand navigation menu.
2. Click **Write a function** (or **Create service** / **Create function**).
3. Fill in the **Basic Configuration** fields:
  - **Deployment type:** Choose *Use an inline editor to create a function*.
  - **Service Name:** Enter a descriptive, lowercase name (e.g., gcs-to-bq-loader).
  - **Region:** Select a region close to your bucket and dataset (e.g., asia-south1).
  - **Runtime:** Select Python 3.11.

### Step 4: Configure Trigger, Authentication, and Scaling

1. **Add Trigger:**
  - Click + **Add Trigger** and select **Cloud Storage** via Eventarc.
  - Set the event type to google.cloud.storage.object.v1.finalized.
  - Select the bucket you created in Step 2.
2. **Authentication:**
  - Choose **Require authentication** for internal and data pipeline workloads.
3. **Scaling & Resources:**
  - Set **Minimum instances** to 0 (enables scale-to-zero so you incur \$0 cost when idle).
  - Set **Maximum instances** to 10 (acts as a cost ceiling).
  - Under **Security**, assign the runtime service account created in Step 2.

### Step 5: Write the Code in the Source Editor

In the inline source editor tab, configure two files and the entry point name:

- **Function entry point:** Set the field value to `load_gcs_to_bigquery`.

- `requirements.txt`:

```
functions-framework==3.*
google-cloud-storage==2.*
google-cloud-bigquery==3.*
```

- `main.py`:

Python

```
import os
import logging
import functions_framework
from google.cloud import bigquery

# Initialize the BigQuery client outside the handler for reuse
bq_client = bigquery.Client()

DATASET_ID = os.environ.get("DATASET_ID", "sales_data")
TABLE_ID = os.environ.get("TABLE_ID", "orders")

@functions_framework.cloud_event
def load_gcs_to_bigquery(cloud_event):
    """Triggered by a Cloud Storage object finalization event."""
    data = cloud_event.data
    bucket_name = data["bucket"]
    file_name = data["name"]

    logging.info(f"Received file: gs://{bucket_name}/{file_name}")

# Validate file extension
```

```

if not file_name.endswith(".csv"):
    logging.warning(f"Skipping non-CSV file: {file_name}")
    return "Skipped"

uri = f"gs://{bucket_name}/{file_name}"
table_ref = f"{bq_client.project}.{DATASET_ID}.{TABLE_ID}"

# Configure the load job to append records
job_config = bigquery.LoadJobConfig(
    autodetect=True,
    source_format=bigquery.SourceFormat.CSV,
    skip_leading_rows=1,
    write_disposition=bigquery.WriteDisposition.WRITE_APPEND
)

try:
    load_job = bq_client.load_table_from_uri(
        uri, table_ref, job_config=job_config
    )
    load_job.result() # Wait for completion
    logging.info(f"Successfully loaded {uri} into {table_ref}")
    return "Success"
except Exception as exc:
    logging.error(f"Failed to load data: {str(exc)}")
    raise exc

```

## Step 6: Deploy and Verify

1. Click **Save and redeploy** (or **Create**) at the bottom of the page.
2. Wait for the build process to finish until the status shows a green checkmark.
3. **Test Execution:**
  - Go to **Cloud Storage** and upload a valid test CSV file (e.g., orders.csv) to your raw bucket.

#### 4. Check Logs & Destination:

- Open the **Observability** tab of your function in the console to inspect execution logs.
- Open **BigQuery** to verify that the table orders has been created and rows were loaded successfully.

## 28. Testing Scenarios

| Test                        | Expected result                                                                   |
|-----------------------------|-----------------------------------------------------------------------------------|
| Valid CSV uploaded          | Function executes and BigQuery load succeeds.                                     |
| Unsupported file extension  | Function rejects or skips the file according to business rules.                   |
| Malformed CSV               | Function records an error and does not silently load bad data.                    |
| Missing BigQuery permission | Function fails with a clear authorization error in logs.                          |
| Missing object permission   | Function cannot read the file and logs the access error.                          |
| Duplicate event/file        | Pipeline behavior should be defined so duplicates do not corrupt downstream data. |

## 29. Troubleshooting Guide

| Problem                               | Possible cause                                       | What to check                                                           |
|---------------------------------------|------------------------------------------------------|-------------------------------------------------------------------------|
| Function does not deploy              | Invalid runtime, source, IAM, API, or configuration. | Deployment output, enabled APIs, runtime support, permissions.          |
| Function deploys but does not trigger | Trigger/event filter mismatch.                       | Trigger configuration, bucket name, event type, Eventarc configuration. |
| 403 / permission denied               | Missing IAM permission.                              | Function service account and required roles.                            |

|                                    |                                                                 |                                                                                        |
|------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------|
| BigQuery load fails                | Schema, format, location, table, or IAM issue.                  | BigQuery error message, destination table, schema, service account.                    |
| Out of memory                      | Workload requires more RAM.                                     | Memory usage and file size; move large processing to Dataflow/Dataproc if appropriate. |
| Timeout                            | Workload is too long for the selected request model.            | Timeout, processing design, asynchronous architecture.                                 |
| Cold start / slow first request    | Instance initialization.                                        | Startup work, dependencies, min instances, startup CPU boost.                          |
| Duplicate data                     | Repeated events/retries or non-idempotent loading.              | Event IDs, object identity, deduplication and write strategy.                          |
| Public access unexpectedly enabled | Authentication configuration allows unauthenticated invocation. | Authentication and IAM invocation permissions.                                         |

## Conclusion

Google Cloud Functions is useful for lightweight, event-driven workloads where code should run automatically in response to requests or Google Cloud events. For a Data Engineer, one of the most important patterns to understand is Cloud Storage → Cloud Run function → BigQuery, together with IAM, logging, retries, idempotency, scaling, and error handling. As workloads become larger or more complex, the correct architecture may involve Dataflow, Dataproc, Cloud Run Jobs, or Cloud Composer instead.

**Documentation note:** This consolidated document is based on the uploaded Google Cloud Functions documentation and adds practical project, testing, troubleshooting, interview, and best-practice sections. Platform limits, pricing, supported runtimes, and UI labels should be checked against current Google Cloud documentation before production deployment.

# Production-Ready Completion Guide

This addendum follows the same progression used in the Managed Airflow guide: introduction → architecture → core concepts → UI/setup → configuration → security → monitoring → troubleshooting → performance/cost → reliability → production design → end-to-end scenario → common mistakes → operational checklist.

## A. Standard learning path

| Stage                   | Expected outcome                                     |
|-------------------------|------------------------------------------------------|
| 1. Introduction         | Explain the service, problem and role.               |
| 2. Architecture         | Explain components and flow.                         |
| 3. Core concepts        | Explain hierarchy, execution and key terminology.    |
| 4. UI and setup         | Create resources and recognize important settings.   |
| 5. Configuration        | Configure runtime, identity, networking and scaling. |
| 6. Security             | Apply IAM, encryption, secrets and least privilege.  |
| 7. Monitoring           | Use logs, metrics and alerts.                        |
| 8. Troubleshooting      | Move from symptom to evidence to root cause.         |
| 9. Performance and cost | Relate design to throughput, latency and cost.       |
| 10. Reliability         | Handle retries, duplicates and recovery.             |
| 11. Production design   | Apply environment separation, CI/CD and ownership.   |
| 12. End-to-end scenario | Explain the service in a realistic pipeline.         |

## 31. Production architecture and idempotency

Cloud Run functions are event-driven compute. Production design should assume that events can be retried and that the same business event may need safe reprocessing.

- Make handlers idempotent where possible.
- Define duplicate-file/event behavior.
- Separate trigger handling from long-running processing that belongs in services such as Dataflow or Cloud Run Jobs.

## 32. Security, secrets and networking

Use a dedicated runtime identity and keep credentials out of source code.

- Use least-privilege service-account permissions.
- Use Secret Manager for secrets instead of hard-coded values.
- Document VPC connectivity requirements when accessing private resources.
- Review ingress/authentication settings for HTTP functions.

## 33. Observability and retries

Operational readiness requires more than a successful deployment.

- Monitor invocation count, errors, latency and resource usage.
- Use structured logging and meaningful error messages.
- Understand trigger retry behavior before deciding whether to retry in application code.
- Route permanently bad events to an appropriate failure/quarantine path where supported by the architecture.

## 34. Deployment and release management

Production functions should be deployed repeatably and changes should be traceable.

- Keep source and dependency files in version control.
- Test before production deployment.
- Use separate environments or projects where appropriate.
- Review runtime support/lifecycle before selecting a runtime.

## **35. Performance and cost**

Serverless scaling can create many concurrent instances if downstream systems cannot absorb the load.

- Set scaling/concurrency controls according to workload behavior.
- Protect databases and APIs from bursts.
- Measure cold-start and execution latency where it matters.
- Use minimum instances only when latency requirements justify the cost.

## **36. Common production mistakes**

- Assuming an event is delivered exactly once.
- Using broad IAM permissions.
- Hard-coding credentials.
- Putting heavy batch processing inside a function.
- Ignoring downstream rate limits.
- Treating logs as the only monitoring mechanism.

## **37. Operational checklist**

- Correct project, region and runtime.
- Trigger/event type verified.
- Authentication/ingress policy verified.
- Runtime service account is least privilege.
- Secrets are externalized.
- Logs and metrics are monitored.
- Retry/idempotency behavior is documented.
- Scaling limits match downstream capacity.
- Deployment artifact is version controlled.
- Rollback/recovery plan is documented.

## **Official references**

<https://docs.cloud.google.com/functions/docs>

<https://docs.cloud.google.com/run/docs/functions/comparison>

<https://docs.cloud.google.com/functions/docs/release-notes>

## **Final learning outcome**

After completing this document, the learner should be able to explain Cloud Run functions, create or configure its main resources, operate it from the Google Cloud console, troubleshoot common failures, and describe production controls.